



School of Computer Science and Statistics

Final Assignment

CS7CS2 – Optimisation for Machine Learning

Lecturer: Dr. Mouli Chakraborty

Contact: mchakrab@tcd.ie

General Guidelines

- You may discuss ideas with classmates; however, **all written work and code must be entirely your own**. Do not share code. All submissions will be checked for plagiarism.
- Reports must be typed and submitted as a **single PDF along with a single .ipynb or .py file via Blackboard**. Do not submit the report inside a zip file.
- **Important:** Marks are awarded primarily for explanation, reasoning, and interpretation, not for raw code or numerical output.
 - Code-based results must be explained.
 - Numerical results must be interpreted.
 - Statements such as “see code” or unexplained plots/numbers are insufficient.
- All plots must have labelled axes, readable text, and a clear written explanation of what they show.

Submission Rules

Submit:

- A short PDF report with codes, plots and brief explanations (WITHIN 50 PAGES).
 - A .ipynb or .py file containing the executable Python code.
-

Benchmark Problems

Use the following benchmark problems throughout the assignment.

Benchmark A: Linear Regression Quadratic Loss

Generate a synthetic dataset with $m = 1000$ samples and two parameters. Let

$$\theta^* = \begin{bmatrix} 3 \\ 4 \end{bmatrix}, \quad x^{(i)} \in \mathbb{R}^2, \quad y^{(i)} = \theta^{*T} x^{(i)} + \varepsilon^{(i)},$$

where $\varepsilon^{(i)}$ is zero-mean Gaussian noise.

Define the loss function by

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left(\theta^T x^{(i)} - y^{(i)} \right)^2.$$

Benchmark B: Toy Neural Network Quadratic Loss

Use the following two-dimensional nonlinear loss:

$$f(x_1, x_2) = (x_1 - 1)^2 + 5(x_2 - 2)^2 + \sin(x_1).$$

Benchmark C: Rosenbrock Function

Use the Rosenbrock function

$$f(x_1, x_2) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2.$$

Benchmark D: Constrained Feasible Set

For constrained optimisation questions, use

$$\mathcal{X} = \{x \in \mathbb{R}^2 : 0.5 \leq x_1 \leq 5, -5 \leq x_2 \leq 10\}.$$

Question 1

[30 marks]

Apply the following methods to Benchmark A, Benchmark B, and Benchmark C, and compare their performance.

(I) **Polyak Step Size**: Implement Gradient Descent using the Polyak step size

$$\alpha_k = \frac{f(x_k) - f^*}{\|\nabla f(x_k)\|^2 + \varepsilon}.$$

Use the following parameter settings: $f^* = 0$, $\varepsilon = 10^{-4}$ for Linear Regression and Toy Neural Network, $\varepsilon = 10^{-3}$ for Rosenbrock, number of iterations = 120.

(II) **Adagrad**: Implement Adagrad using per-coordinate adaptive step sizes.

Use the following initial learning rates: Linear Regression: $\alpha_0 = 1.8$, Toy Neural Network: $\alpha_0 = 1.2$, Rosenbrock: $\alpha_0 = 0.45$, $\varepsilon = 10^{-5}$, number of iterations = 120.

(III) **RMSprop** : Implement RMSprop using exponentially weighted squared gradients.

Use the following settings: Linear Regression: $\alpha_0 = 0.22$, Toy Neural Network: $\alpha_0 = 0.14$, Rosenbrock: $\alpha_0 = 0.0035$, $\beta = 0.9$, $\varepsilon = 10^{-5}$, number of iterations = 120.

(IV) **Polyak Momentum / Heavy Ball** : Implement

$$z_{k+1} = \beta z_k + \alpha \nabla f(x_k), \quad x_{k+1} = x_k - z_{k+1}.$$

Use the following settings:

Linear Regression: $\alpha = 0.045$, $\beta = 0.88$, Toy Neural Network: $\alpha = 0.035$, $\beta = 0.90$, Rosenbrock: $\alpha = 0.0008$, $\beta = 0.86$, number of iterations = 120.

In addition, use a constant step-size Gradient Descent baseline with: Linear Regression: $\alpha = 0.08$, Toy Neural Network: $\alpha = 0.06$, Rosenbrock: $\alpha = 0.0012$.

- Tasks:** (a) Implement all four methods.
 (b) Plot objective value versus iteration for each benchmark.
 (c) For Benchmark B and Benchmark C, produce contour plots with optimisation trajectories.
 (d) Plot the adaptive step-size evolution for Polyak, Adagrad, and RMSprop.
 (e) Compare convergence speed, smoothness, and stability.
-

Question 2

[30 marks]

Apply the following methods and compare their behaviour on the benchmark problems.

(I) Nesterov Momentum / Acceleration

Implement

$$z_{k+1} = \beta_k z_k - \alpha \nabla f(x_k + \beta_k z_k), \quad x_{k+1} = x_k + z_{k+1},$$

with

$$\beta_k = \min \left(\frac{k-1}{k+2}, \beta_{\max} \right).$$

Use the following settings: Linear Regression: $\alpha = 0.06$, $\beta_{\max} = 0.90$, Toy Neural Network: $\alpha = 0.035$, $\beta_{\max} = 0.92$, Rosenbrock: $\alpha = 0.0007$, $\beta_{\max} = 0.90$, number of iterations = 150.

(II) Adam: Implement Adam using first and second moments.

Use the following settings: Linear Regression: $\alpha = 0.12$, $\beta_1 = 0.82$, $\beta_2 = 0.999$, Toy Neural Network: $\alpha = 0.08$, $\beta_1 = 0.80$, $\beta_2 = 0.999$, Rosenbrock: $\alpha = 0.006$, $\beta_1 = 0.80$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, number of iterations = 150.

(III) Mini-Batch Stochastic Gradient Descent (SGD): Use Benchmark A only.

Use the following settings: learning rate $\alpha = 0.06$, batch sizes $b = 5$ and $b = 40$, epochs = 50, shuffle data at every epoch.

(IV) SGD with Noise: Use Benchmark A only.

Repeat Part (c), but increase the output noise by an additional factor of 6.0.

Use a constant-step Gradient Descent baseline with:

Linear Regression: $\alpha = 0.08$, Toy Neural Network: $\alpha = 0.06$, Rosenbrock: $\alpha = 0.0012$.

Tasks:

- (a) Compare Nesterov, Adam, and the constant baseline on all three benchmarks.
 - (b) Plot objective value versus iteration.
 - (c) Plot contour trajectories for Benchmark B and Benchmark C.
 - (d) For SGD, compare the effect of batch size.
 - (e) For noisy SGD, analyse how additional noise changes convergence.
-

Question 3

[30 marks]

Newton's Method and Local Approximation

(I) First-order Local Approximation

Use a one-dimensional test function

$$g(x) = x^4.$$

Construct and plot its first-order approximation around $x_0 = 0.25$:

$$g(x) \approx g(x_0) + g'(x_0)(x - x_0).$$

(II) Second-order Local Approximation

Using the same function, construct the second-order approximation:

$$g(x) \approx g(x_0) + g'(x_0)(x - x_0) + \frac{1}{2}g''(x_0)(x - x_0)^2.$$

Use this to derive the Newton update for the one-dimensional case.

Then apply Newton's Method to Benchmark A, Benchmark B, and Benchmark C.

Use the following parameter settings:

(A) Gradient Descent baseline: Linear Regression: $\alpha = 0.08$, Toy Neural Network: $\alpha = 0.06$, Rosenbrock: $\alpha = 0.001$.

(B) Newton step: Linear Regression: $\alpha = 1.0$, Toy Neural Network: $\alpha = 0.85$, Rosenbrock: $\alpha = 0.22$.

(C) Gradient Descent iterations = 80, Newton iterations = 20, Hessian damping = 10^{-8} .

Tasks:

- (a) Plot the original one-dimensional function together with its first-order and second-order approximations.
 - (b) Implement Newton's Method on all three benchmarks.
 - (c) Compare Newton's Method with Gradient Descent.
 - (d) Plot contour trajectories for Benchmark B and Benchmark C.
 - (e) Plot the update magnitude versus iteration for both methods.
-

Question 4

[30 marks]

Derivative Approximation and Derivative-Free Optimisation

Part A: Finite Difference Approximation and Nesterov Random Search

Use Benchmark B.

(I) Finite Difference Approximation

Implement forward finite-difference gradient approximation

$$\frac{\partial f}{\partial x_i}(x) \approx \frac{f(x + \delta e_i) - f(x)}{\delta}$$

for two values of δ . **Use:** good setting: $\alpha = 0.08$, $\delta = 0.05$, poor setting: $\alpha = 0.08$, $\delta = 0.8$, iterations = 120.

(II) **Nesterov Random Search**

Implement

$$x_{k+1} = x_k - \alpha \frac{f(x_k + \delta u_k) - f(x_k)}{\delta} u_k$$

where u_k is a random unit vector. **Use:** $\alpha = 0.025$, $\delta = 0.08$, iterations = 220.

Use exact Gradient Descent as a baseline with $\alpha = 0.06$.

Part B: Derivative-Free Methods

Use Benchmark C.

(I) **Simplex Method (Nelder–Mead)** : Implement the Nelder–Mead simplex algorithm. **Use:** initial simplex step = 0.35, iterations = 160.

(II) **Grid Search:** Implement a brute-force grid search over a bounded region. **Use:** search region: $x_1 \in [-2, 2]$, $x_2 \in [-1, 3]$, grid size 55×55 .

Tasks:

- (a) Compare exact GD, finite-difference GD, and Nesterov random search on Benchmark B.
- (b) Plot objective value versus iteration.
- (c) Plot contour trajectories for Benchmark B.
- (d) Plot the Nelder–Mead trajectory on Benchmark C.
- (e) Plot the grid-search sampling pattern and best-so-far value on Benchmark C.
- (f) Compare computational behaviour and accuracy.

Question 5

[30 marks]

Constrained Optimisation: Projected Gradient Descent and Penalty Methods

Use Benchmark B with the constraint

$$x_1 \geq 0.5.$$

Start from the infeasible initial point

$$x_0 = (0.2, 4.0).$$

(I) **Projected Gradient Descent:** Implement projected gradient descent using the projection

$$x_1 \leftarrow \max(0.5, x_1).$$

Use: learning rate $\alpha = 0.08$, iterations = 100.

(II) **Penalty Method:** Use the penalised objective

$$F_\lambda(x) = f(x) + \lambda \max(0, -x_1 + 0.5).$$

Use: learning rate $\alpha = 0.05$, $\lambda = 0.15, 1.8, 4.5$, for $\lambda = 4.5$, reduce the step size to $\alpha = 0.03$, iterations = 100. Also use unconstrained Gradient Descent with: $\alpha = 0.07$.

Tasks:

1. Implement unconstrained GD, projected GD, and penalty GD.
2. Plot objective or penalised objective versus iteration.
3. Produce a contour plot with the feasible boundary and optimisation paths.
4. Plot the constraint violation $\max(0, 0.5 - x_1)$ on a logarithmic scale.
5. Produce an additional zoomed-in constraint violation plot for the first 30 iterations.
6. Compare how quickly each method enforces feasibility.

Question 6**[30 marks]****Linear Programmes and Frank–Wolfe Algorithm**

Use the feasible set

$$\mathcal{X} = \{x \in \mathbb{R}^2 : 0.5 \leq x_1 \leq 5, -5 \leq x_2 \leq 10\}.$$

(I) Linear Programme

Consider the linear objective

$$f(x) = a^T x, \quad a = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Solve the linear programme over $\mathcal{X}$.**(II) Frank–Wolfe for an Interior Optimum**

Use the nonlinear objective

$$f(x_1, x_2) = (x_1 - 1)^2 + (x_2 - 5)^2.$$

Implement Frank–Wolfe:

$$z_k = \arg \min_{x \in \mathcal{X}} \nabla f(x_k)^T x, \quad x_{k+1} = \beta x_k + (1 - \beta) z_k.$$

Use: initial point $x_0 = (1, 1)$, $\beta = 0.90$ and $\beta = 0.985$, iterations = 180.**(III) Frank–Wolfe for a Boundary Optimum**

Use

$$f(x_1, x_2) = x_1^2 + x_2^2.$$

Since $x_1 \geq 0.5$, the constrained optimum lies on the boundary. **Use:** initial point $x_0 = (3, 3)$, $\beta = 0.93$, iterations = 140.**Tasks:**

1. Plot the feasible region and the contours of the linear objective.
2. Plot the Frank–Wolfe convergence curves for the interior-optimum case.
3. Produce contour plots showing the Frank–Wolfe trajectories.
4. Plot the evolution of x_k and z_k versus iteration.
5. Compare the behaviour when the optimum lies in the interior and when it lies on the boundary.

All the Best !